Introduction

Tree

Data Structures and its Applications (UE25CS52A)

N S Kumar
Professor
Department of Computer Science & Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree

1. Search on a binary search tree in the worst case will do

a) n comparisons

b) n/2 comparisons

c) Log(n) to base 2 comparisons

d) Log(n) to base e comparisons

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree

1. Search on a binary search tree in the worst case will do

a) n comparisons

b) n/2 comparisons

c) Log(n) to base 2 comparisons

d) Log(n) to base e comparisons

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree

2. The smallest key in a binary search tree will

a) Always be a leaf in the left subtree

b) Always be a leaf in the right subtree

c) Could be an internal node with no left subtree

d) Could be an internal node with no right subtree

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree

2. The smallest key in a binary search tree will

a) Always be a leaf in the left subtree

b) Always be a leaf in the right subtree

c) Could be an internal node with no left subtree

d) Could be an internal node with no right subtree

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree

3. A binary tree has $n$nodes. What is the maximum possible number of nodes at level k, assuming the root is at level 0?

a) k
b) $2k$
c) 2^k
d) 2^{k+1}

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree

3. A binary tree has $n$nodes. What is the maximum possible number of nodes at level k, assuming the root is at level 0?

a) k
b) 2k
c) 2^k
d) 2^{k+1}

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree

4. A binary tree contains 15 nodes. What is the minimum possible height of the tree, where the height of a single-node tree is 0?

a) 2
b) 3
c) 4
d) 14

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree

4. A binary tree contains 15 nodes. What is the minimum possible height of the tree, where the height of a single-node tree is 0?

a) 2
b) 3
c) 4
d) 14

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree

5. Which statement is always true for a binary tree?

a) Every node has exactly two children
b) Every leaf has two children
c) A node can have at most two children
d) Every internal node has two children

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz : Tree

5. Which statement is always true for a binary tree?

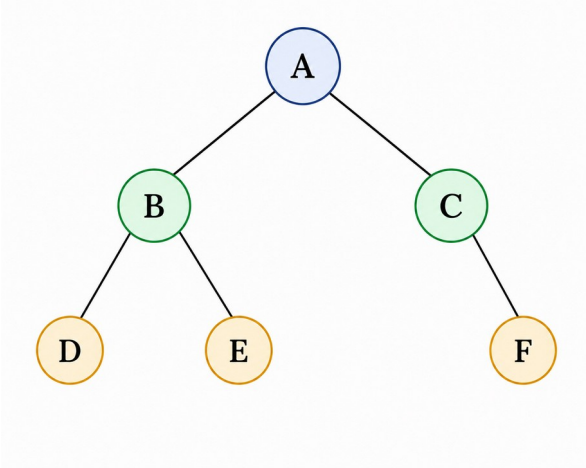
a) Every node has exactly two children
b) Every leaf has two children
c) A node can have at most two children
d) Every internal node has two children

© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree

Definitiion

- A **binary tree** is a tree in which every node has **at most two children**.
- The two children are distinguished as the **left child** and **right child**.
- A binary tree is either:
  - An **empty tree**, or
  - A root node together with two disjoint binary trees called its:
    - **left subtree**
    - **right subtree**



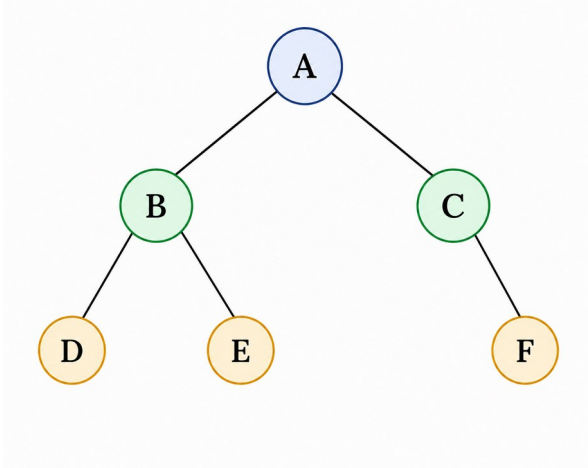
© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree

Inorder traversal of the binary tree

- If tree is not empty
  - Visit the left subtree in inorder
  - Visit the root
  - Visit the right subtree in inorder

Inorder traversal of this tree:
D B E A C F



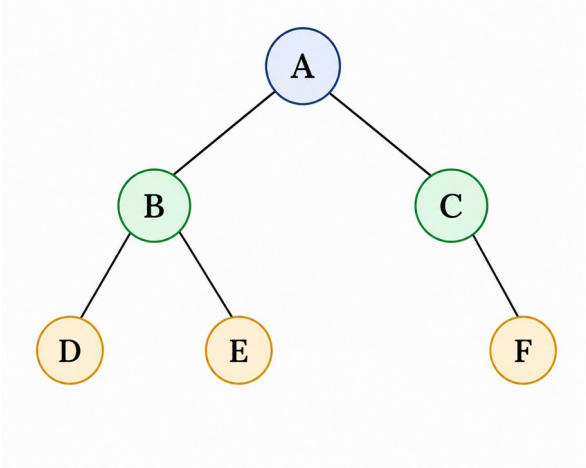
© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree

preorder traversal of the binary tree

- If tree is not empty
  - Visit the root
  - Visit the left subtree in preorder
  - Visit the right subtree in preorder

preorder traversal of this tree:
A B D E C F

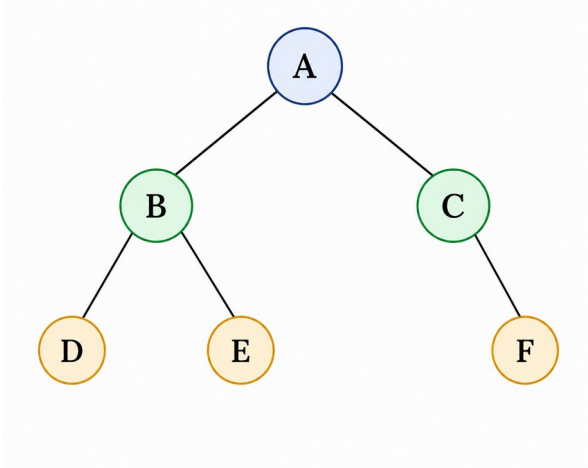


© Confidential Information of PES University. All Rights Reserved - 2026

postorder traversal of the binary tree

- If tree is not empty
  - Visit the left subtree in postorder
  - Visit the right subtree in postorder
  - Visit the root

postorder traversal of this tree:
D E B F C A



© Confidential Information of PES University. All Rights Reserved - 2026

Binary Search Tree

- Every node has a key
- Nodes are inserted based on the key
- If the key of a node is less than that of the root, it will be inserted in the left subtree
- Otherwise, it will be inserted in the right subtree

© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree

Binary Search Tree : simple and small interface

- Create a tree
- Destroy the tree
- Insert the key
- Display the tree in inorder

- Find the number of nodes
- Find the number of leaves

© Confidential Information of PES University. All Rights Reserved - 2026

Binary Search Tree : structure of the functions

- Recursive on nodes and not on tree
- Model for recursive routine :
  - Empty tree
  - Tree with left and right subtrees

© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree

Binary Search Tree : interface

void init(tree_t *ptr_tree);
    Ground the root.
void deinit(tree_t *ptr_tree);
    Walk through the tree
    Remove the left subtree
    Remove the right subtree
    Remove the root
void disp(tree_t *ptr_tree);
    // This is inorder traversal of the tree
    If the tree is not empty
        Display the left subtree
        Display the root
        Display the right subtree

void insert(tree_t *ptr_tree, int key);
    If the tree is empty, make this the root
    Otherwise compare the keys
    Decide to insert in the left subtree or the right subtree

© Confidential Information of PES University. All Rights Reserved - 2026

Binary Search Tree : insert

```
// on the tree; non recursive
void insert(tree_t *ptr_tree, int key)
{
        node_t *temp = (node_t*)malloc(sizeof(node_t));
        temp->key_ = key;
        temp->left_ = temp->right_ = NULL;
        ptr_tree->root_ = insert_recursive(ptr_tree->root_, temp);
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Binary Search Tree : insert

```
// on the node; recursive; observe the assignment to the root
static node_t* insert_recursive(node_t* root, node_t* temp)
{
        if(root == NULL)
        {
                root = temp;
        }
        else if(root->key_ > temp->key_)
        {
                root->left_ = insert_recursive(root->left_, temp);
        }
        else
        {
                root->right_ = insert_recursive(root->right_, temp);
        }
        return root;
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree : some recursive algorithms on the node

1. Count the number of nodes

```
int number_of_nodes_recursive(node_t* temp)
{
        if(temp == NULL)
        {
                return 0;
        }
        else
        {
                return 1 + number_of_nodes_recursive(temp->left_)
                        + number_of_nodes_recursive(temp->right_);
        }
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree : some recursive algorithms on the node

2. Count the number of leaves

```
int number_of_leaves_recursive(node_t* temp)
{
        if(temp == NULL)
        {
                return 0;
        }
        else if(temp->left_ == NULL && temp->right_ == NULL)
        {
                return 1;
        }
        else
        {
                return number_of_leaves_recursive(temp->left_)
                        + number_of_leaves_recursive(temp->right_);
        }
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree : some recursive algorithms on the node

3. Compare two trees

```
int are_same_recursive(node_t* root1, node_t* root2)
{
        if(root1 == NULL && root2 == NULL)
                return 1;
        else if(root1 == NULL || root2 == NULL)
                return 0;
        else
                return root1->key_ == root2->key_ &&
                        are_same_recursive(root1->left_, root2->left_) &&
                        are_same_recursive(root1->right_, root2->right_);
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree : some recursive algorithms on the node

4. Copy tree

```
node_t* copy_recursive(node_t* ptr_src)
{
        node_t *temp;
        if(ptr_src != NULL)
        {
                temp = (node_t*)malloc(sizeof(node_t));
                temp->key_ = ptr_src->key_;
                temp->left_ = copy_recursive(ptr_src->left_);
                temp->right_ = copy_recursive(ptr_src->right_);
                return temp;
        }
        else
        {
                return NULL;
        }
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Binary search Tree : some recursive algorithms on the node

5. search

```
node_t* search_recursive(node_t* root, int key)
{
        if(root == NULL)
                return NULL;
        else if(key == root->key_)
                return root;
        else if(key < root->key_)
                return search_recursive(root->left_, key);
        else
                return search_recursive(root->right_, key);

}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Binary search Tree : some recursive algorithms on the node

6. Inorder successor if the argument is the right subtree of a given node

```
node_t* inorder_successor(node_t* root)
{
        node_t* temp = root;
        while(temp->left_ != NULL)
                temp = temp->left_;
        return temp;
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Binary search Tree : some recursive algorithms on the node

7. Delete based on key

```
node_t* delete_key_recursive(node_t* root, int key)
{
    if(root == NULL) // empty tree
   {
                return NULL;
   }
   else if(key < root->key_) // search in left subtree
   {
      root->left_ = delete_key_recursive(root->left_, key);
   }
   else if (key > root->key_) // search in right subtree
   {
      root->right_ = delete_key_recursive(root->right_, key);
   }
  else  // match found; action of deletion should start
```


© Confidential Information of PES University. All Rights Reserved - 2026

Binary search Tree : some recursive algorithms on the node

7. Delete based on key continued

```
if(root->left_ == NULL && root->right_ == NULL) // 1 leaf
{
        free(root);    return NULL;
}
else if(root->left_ == NULL) // 2. no left
{
        node_t* temp = root->right_; free(root);
        return temp;
}
else if(root->right_ == NULL) // 3. no right
{
        node_t* temp = root->left_;  free(root);
        return temp;
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Binary search Tree : some recursive algorithms on the node

7. Delete based on key continued; we have found a match

```
                // 4. has both left and right
                // find the inorder successor
                else
                {
                        node_t* temp = inorder_successor(root->right_);
                        root->key_ = temp->key_; // replace
                        // search with the new key in the right subtree
                        root->right_ = delete_key_recursive(root->right_, temp->key_);
                }
        }
        return root;
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Binary Tree : persistence

- A single traversal will not define the tree uniquely
- An inorder and a postorder traversal combined define a tree uniquely
- Rightmost node in the postorder traversal will be the root of the tree
- This node in the order traversal divides the tree in left subtree and the right subtree
- Repeating traversal from right to left of post order traversal and locating the node
  to divide the inorder traversal to left and right subtrees
  will recreate the tree

© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

N S Kumar
Professor

Department of Computer Science & Engineering (AI & ML)